

Steady State Cost in Competing Aggregation-Fragmentation Dynamics

Fitzwilliam James Keenan-Koch^{1,*}

¹Vermont Complex Systems Institute, University of Vermont, Burlington, VT, USA

*fkeenank@uvm.edu

ABSTRACT

Aggregation and fragmentation among polymers has long been an area of active research and utility in physical chemistry. Drawing inspiration from the dynamics of social formulation, we make a preliminary effort at minimizing cost and probability of domination among various potential fragmentation kernels. This work has relevance for any work formalizing the formulation, dissolving, and regulation of groups of abstract individuals. Social groups are of particular interest, from musical subcultures to the regulation of corporate consolidation.

1 Introduction

Things fall apart, Yeats wrote¹. Of course, they also come together. What balance is struck between the two, and crucially, **what does this balance cost?**

To attempt an answer to this question, we turn to the physics of aggregation and fragmentation, the competing processes by which particles combine to form larger clusters and break up into smaller ones. As thoroughly explicated in *A Kinetic View of Statistical Physics*, aggregation and fragmentation can be thought of as an infinite set of differential equations over all cluster sizes from one to infinity². Rates of change are based on current amounts of clusters and a kernel specifying reaction rates between clusters of various sizes. Clusters of size one are termed monomers, and the formulation as a whole is termed the Smoluchowski equations.

While Krapivsky *et al* term such differential equations master equations, their equations make a mean-field assumption: they track the *expected* cluster size distribution and assume it deterministic rather than keeping track of the full spread of probability over the stochastic evolution of the system. In other words, they track the *count* of the cluster sizes, rather than the probability of *each count* of each cluster.

A more complete accounting is given by the Marcus–Lushnikov process, in which $ML^{(n)}(x, t)$ is a random number whose probability evolves according to a Markov Chain involving all other $ML^{(n_i)}(x, t)$ ³. These models typically rescale the total mass of the system to be 1, and are identical to the Smoluchowski equations if the kernel is homogenous, or as M , the number of initial unitary particles considered, goes to infinity. However, these models are far less tractable, and even in 2018 computing the time-evolving probability with only 48 initial monomers took around 3000 minutes⁴.

Regardless of the form, prior work has largely focused on either the time-evolution of cluster sizes or the static steady state distribution of them. Instead, this work focus on the interplay between the steady state rate of fragmentation, or the *cost* of maintaining the steady state, and the distribution of cluster sizes. More simply, using the Smoluchowski equation form, this work charts the pareto front between minimizing the steady state probability of having an oversized *dominant* cluster and minimizing the number of units of mass continually split by fragmentation.

2 Methods:

The problem

We model competing aggregation and fragmentation over a variety of rate kernels in order to study the steady-state distribution of cluster sizes as well as the steady-state rate of fragmentation measured in units of mass removed from larger clusters. Our question is simple: given a pattern of aggregation, what fragmentation technique minimize the proportion of dominant clusters and steady-state fragmentation rate? Does this dual-minimization create a pareto front?

Preliminaries

Aggregation

Aggregation is generally the transition

$$c_i + c_j \xrightarrow{K_{ij}} c_{i+j}$$

where c_i denotes a cluster of mass i , $K_{ij} = K_{ji}$ is the symmetric $M \times M$ aggregation rate matrix between clusters of mass i and j , and N is the total mass normalized to the monomer size. The clusters $\vec{c}$ are assumed well-mixed, so encounter rates depend only on cluster quantities.

Then the continuous ordinary differential equation for all cluster sizes c_k is the sum of aggregation rates producing a cluster size and the rate of aggregation reducing the amount of a cluster size, with each weighted by the current amount of the relevant clusters:

$$\frac{dc_k}{dt} = \frac{1}{2} \sum_{i+j=k} K_{ij} c_i c_j - c_k \sum_{i \geq 1} K_{ik} c_i \quad (1)$$

$$= \frac{1}{2} \sum_{i,j \leq 1} K_{ij} c_i c_j [\delta_{i+j,k} - \delta_{i,k} - \delta_{j,k}] \quad (2)$$

where δ is the Kronecker delta.

Fragmentation

Fragmentation is generally the transition

$$c_{i+j} \xrightarrow{F_{ij}} c_i + c_j$$

where F is a symmetric matrix of size $M \times M$ and F_{ij} is the rate of fragmentation of a cluster of size $i+j$ into a cluster of size i and another of size j . The ordinary differential equation for fragmentation across all clusters is then:

$$\frac{dc_k}{dt} = \sum_{j \geq 1} F_{kj} c_{j+k} - \frac{1}{2} c_k \sum_{i+j=k} F_{ij} \quad (3)$$

Aggregation and Fragmentation

Putting equations (1) and (3) together and scaling them by parameterized rates α and β yields the main differential equation we use in our model:

$$\frac{dc_k}{dt} = \dot{c}_k = \frac{1}{2} \sum_{i+j=k} \alpha K_{ij} c_i c_j - c_k \sum_{j \geq 1} \alpha K_{kj} c_j + \sum_{j \geq 1} \beta F_{kj} c_{j+k} - \frac{1}{2} c_k \sum_{i+j=k} \beta F_{ij} \quad (4)$$

The Model

Using equation (4) as our basis, we let $T = 100$ be the total time the integrator runs, and $M = 100$ be the total amount of mass, which is conserved across all steps. Note that in our calculations we use a normalized representation of mass $N = 1$, so the initial choice of M affects only the speed of our calculation and the resolution of our results; in other words, M is effectually the number of discrete ‘buckets’ of cluster sizes. The system has a complexity of $\mathcal{O}(M^2)$: higher M produces significantly slower, but more granular, integrations. With this formalization in place, we use the `scipy` Python package to integrate the differential equation and thus calculate the time evolution of the system. From this time evolution we are able to easily calculate the earliest time index s at which the state of the system ceases to change and thus reaches a steady state.

Cost

The main emphasis of this work is to keep track of the time evolution not only of the cluster sizes, but of the *rate of fragmentation*. More specifically, we are interested in the rate at which unit masses are broken off of clusters; the number of unit masses that break at each infinitesimal dt . Letting κ indicate this cost, we can write its rate as:

$$\dot{\kappa} = \sum_{k=1}^M \sum_{i+j=k} \beta F_{ij} c_k(t) \times \min(i, j) \quad (5)$$

where we write only one fragmentation sum because the sum of mass leaving all cluster sizes double counts with the sum of mass entering all cluster sizes. To calculate the full time-evolution of cost, we augment the state vector with an additional variable c_κ with associated differential equation (5).

Therefore when the integrator finishes running we have $\vec{\kappa}$, the vector of how κ changes over the full time-evolution of the system. However, what we are principally interested in is $\dot{\kappa}$, the continuous rate; more particularly, we are interested in $\dot{\kappa}_s$, the rate of fragmentation *when the system is in a steady state*. To derive this, we simply calculate the gradient of $\vec{\kappa}$ from $t = s$ onwards using `numpy`.

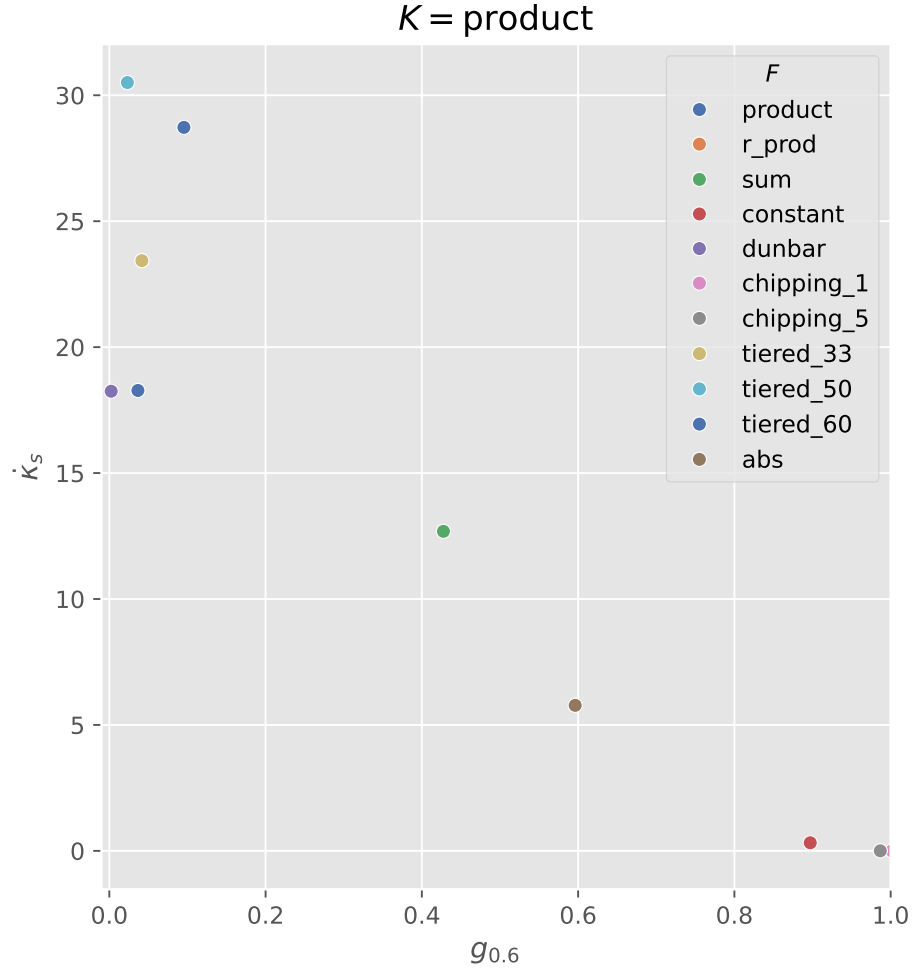


Figure 1. Pseudo-Pareto front for the [Product](#) kernel with all $\alpha = 1$ and $\beta = .01$

Domination

The goal of this model is to simulate certain social systems just as much as it is physical. As such, we are interested not in gelation, when all mass in the system converts to a single cluster, but in **Domination**, when a cluster is larger than the sum of all other clusters. Clearly, this must happen whenever a cluster contains more than 50% of the mass. Generally, however, we are able to say that the proportion of mass in clusters greater than some dominating threshold x is the weighted ratio:

$$g_x(t) = \frac{\sum_{k > xM} c_k(t)k}{\sum_1^M c_k(t)k} \quad (6)$$

Where the denominator is always 1 but is left included for completeness and to account for any numerical drift. To calculate some g_x in practice, we take the distribution of clusters at the steady state $t = s$ and complete equation (6); we are interested in this value only at the steady state of the system and leave this implicit in the notation. Throughout, we report generally on $g_{0.6}$, the proportion of mass in clusters greater than $.6M$, though $g_{0.8}$ is calculated as well.

Initial Conditions

We define three initial conditions $\vec{c}_0$: **All Monomer**, **Di_Monomer**, and **First 10**, where the mass is initially evenly split between the cluster sizes in the condition title. Letting k be the total number of cluster sizes we want, to ensure that the mass sums to one we write this as:

$$c_x(0) = \frac{1}{xk} \quad \forall x \leq k \quad (7)$$

Regardless of the initial condition used, the integration runs in the same fashion and calculates the same values as outlined above.

Kernels

Recall that the only requirement for our kernels K and F is that they are symmetric. It is then easy to see that there is an infinite set of kernels, even without our scaling rates α and β ; consider, for example, $(i + j)^p \forall p \in \mathbb{R}$. For this reason, we use kernels drawn from a combination of physics literature and inspiration from social phenomena. The equation for all kernels is available in Table 1.

First, aggregation. Consider our clusters as polymer chains consisting of k monomers, each with all monomers having the same amount of f -functional reactive endgroups. Clearly, anytime a monomer joins a polymer chain, two endgroups are consumed and f are added, as in equation (8). If all endgroups are equally reactive, then the reaction rate between two clusters of sizes i, j will equal the product of the number of endgroups. Together, we can write this formulation as follows:

$$kf - 2(k - 1) = (f - 2)(k + 2) \quad (8)$$

$$g(i, j) = (f - 2)^2 ij + 2(f - 2)(i + j) + 4 \quad (9)$$

When $f = 2$, clearly equation (9) yields a constant kernel, when $f \rightarrow \infty$, it yields a product kernel, and anywhere in between it is a linear combination of a constant, sum, and product kernel. In social phenomena, we can interpret the number f to indicate *how many social bonds* a person is able to maintain within the social context. In 1992, Dunbar used neocortex size and extrapolation from social dynamics observed in other primates to argue that humans can maintain 150 stable relationships⁵. We use this number to construct the **dunbar** kernel. We might also argue that in the context of a large corporation, for example, the number of functional ‘connections’ *homo corporatum* might have with another is practically infinite; we use this argument to construct the **product** kernel. In practice, the two are functionally identical. For good measure, we throw in the **sum** and **constant** kernels as well. Clearly, moving from the constant to sum to product kernels will accelerate the rate of aggregation, all other things being equal.

Next, fragmentation, again drawing inspiration from both prior physics and social phenomena. For example, the **chipping1** and **chipping5** kernels do exactly what they sounds like: chip away blocks of either one or five masses only. This is a common behavior in chemical polymers, where only the ends of the chain are available for reaction. The **tiered** and **Absolute Difference** kernels are drawn from theories of financial regulation. The **tiered** kernel splits apart clusters at twice the product rate provided they are over a certain threshold. This represents a regulatory body attempting to allow for smaller clusters while preventing larger ones; it is parameterized as θM and test on three separate theta values. The **Absolute Difference** kernel, on the other hand, prioritizes splitting groups evenly. Finally, the **rprod** kernel is the the reverse: the larger the cluster, the less likely the fragmentation, producing something like ‘too big to fail’.

3 Results

Figure 3 in the Appendix shows the comparison plots between κ_s and $g_{0.6}$ for all aggregation kernels. However, **Absolute Difference** and **Reverse Product** are uninteresting, while **Dunbar** and **Product** are functionally identical. Figure 2 therefore drills down into a subset of four kernels with the most activity. In all but the **Product** aggregation kernel, the steady state cost is almost zero across all steady states, and the fragmentary kernel dictates only how strong the dominating mass will be. Through, all three, the **Tiered** kernels perform the best, regardless of θ value, and the two **chipping** kernels the worst.

In the **Product** aggregation kernel, however, we see a different story. Strong tradeoffs between κ_s and $g_{0.6}$, most clearly when $\alpha = 1$ and $\beta = .01$. Figure examines this apparent Pareto front more closely. In other words, a Pareto front of dual optimization emerges most clearly *when the fragmentary kernel is overwhelmed*. Note that we show only the **All Monomer** initial conditions, since the other initial conditions we tested were functionally identical.

4 Discussion

The Discussion should be succinct. It typically involve any big picture discussions not covered in Sec. 3, i.e., the Results section. This was an example of a section reference using the `label` and `ref` commands. Figures and tables can also be referenced in LaTeX using the `label` and `ref` commands, e.g. Fig. ?? and Tab. ??.

This discussion can therefore mention scientific consequences of the results, ideas for future work, stress approximations or simplifications that limit the applicability of the model, or discuss things you tried that did not work. The Discussion section must not contain subheadings.

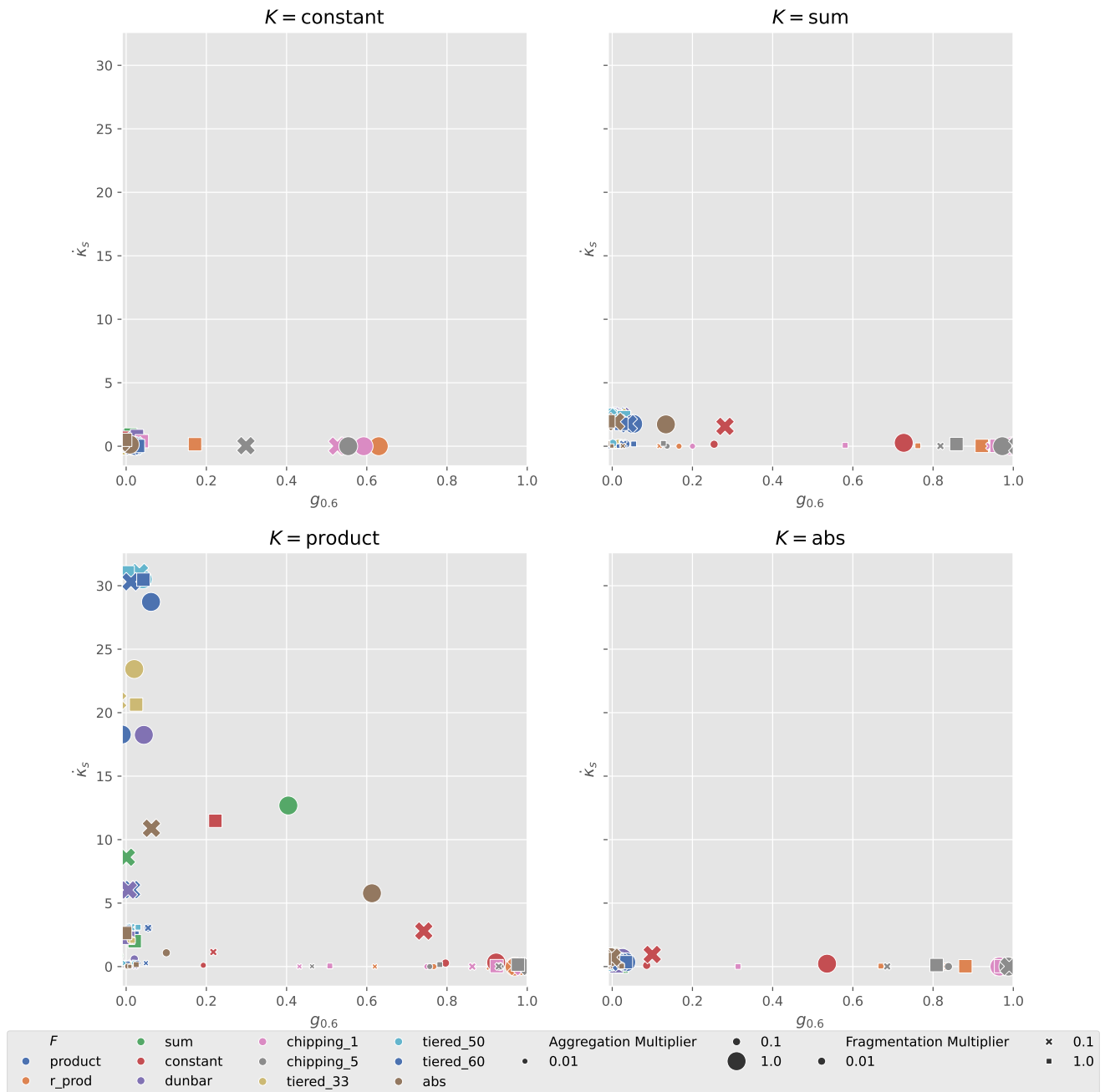


Figure 2. Comparison for four most interesting aggregation kernels with initial condition **First 10**. The size of the node indicates α , while the shape of the node indicates β . Unsurprisingly, larger values of α and smaller values of β generally produce higher values of our two optimizing variables.

Name	Equation
Chipping1	$F_{ij} = \beta(\delta_{i,1} + \delta_{j,1})$
Chipping5	$F_{ij} = \beta(\delta_{i,5} + \delta_{j,5})$
Sum	$F_{ij} = \beta(i + j)$
Product	$F_{ij} = \beta(ij)$
Constant	$F_{ij} = \beta$
Reverse Product	$F_{ij} = \frac{\beta}{ij}$
Dunbar	$F_{ij} = \beta\left(ij + \frac{2(i+j)}{150} + \frac{4}{150^2}\right)$
Absolute Difference	$F_{ij} = \beta i - j .$
Tiered	$F_{ij} = 2\beta ij * 1[(i + j) > \theta M]$ with $\theta = .33$, $\theta = .5$, and $\theta = .6$

Table 1. Specification of the nine kernels used for aggregation and fragmentation. Note that all kernels are also used for aggregation by substituting α for β .

Acknowledgements

Thanks to Laurent Herbert-Dufresne, VSCI in general, and the authors of *A Kinetic View of Statistical Physics*².

Additional information

Data availability statement: No real world data is used in this project

Code availability statement: Code is available at github.com.

References

1. Yeats, W. *The Second Coming* (1919).
2. Krapivsky, P. L., Redner, S. & Ben-Naim, E. *A Kinetic View of Statistical Physics* (Cambridge University Press, 2010), 1 edn.
3. Aldous, D. J. Deterministic and stochastic models for coalescence (aggregation and coagulation): A review of the mean-field theory for probabilists. *Bernoulli* **5**, 3–48 (1999).
4. Manuchehrfar, F., Tian, W., Chou, T. & Liang, J. Evolution of Coagulation-Fragmentation Stochastic Processes Using Accurate Chemical Master Equation Approach. *Commun. Inf. Syst.* **19**, 37–55, DOI: [10.4310/cis.2019.v19.n1.a3](https://doi.org/10.4310/cis.2019.v19.n1.a3) (2019).
5. Dunbar, R. I. M. Neocortex size as a constraint on group size in primates. *J. Hum. Evol.* **22**, 469–493, DOI: [10.1016/0047-2484\(92\)90081-J](https://doi.org/10.1016/0047-2484(92)90081-J) (1992).

Table 2. Model Card**1. Model Details**

The problem	Calculating rate of fragmentation at steady state of fragmenting system.
Modeling approach	Ordinary Differential Equations, inspired by, but not quite, Master Equations
Degrees of freedom	$M - 1$, with M the mass in number of monomers.
Number of parameters	7: $\alpha, \beta, K, F, M, T, \vec{c}(0)$
Implementation language	Python
Contact information	Fitz Koch: fw.j.keenan.koch@gmail.com

2. Intended Use

Primary intended uses	Determining the ‘cost’ of a (Dis)Aggregating system at steady state
Primary users	Applied modelers of competing aggregation and fragmentation
Out-of-scope uses	Prediction and forecasting on more complicated real world scenarios.

3. Key assumptions

Population size	Parameterized, but here 100
Contact structure	Mean-field
Time structure and memory effects	Continuous time

4. Metrics

Computational complexity	$\mathcal{O}(M^2)$
--------------------------	--------------------

5. Suggested next steps

Issues with implementations	None noticed
Recommendations for use	Specify kernels of interest and search over variety of values.
Future work	Compare mean-field values with full consideration of stochastic Master Equations

5 Appendix

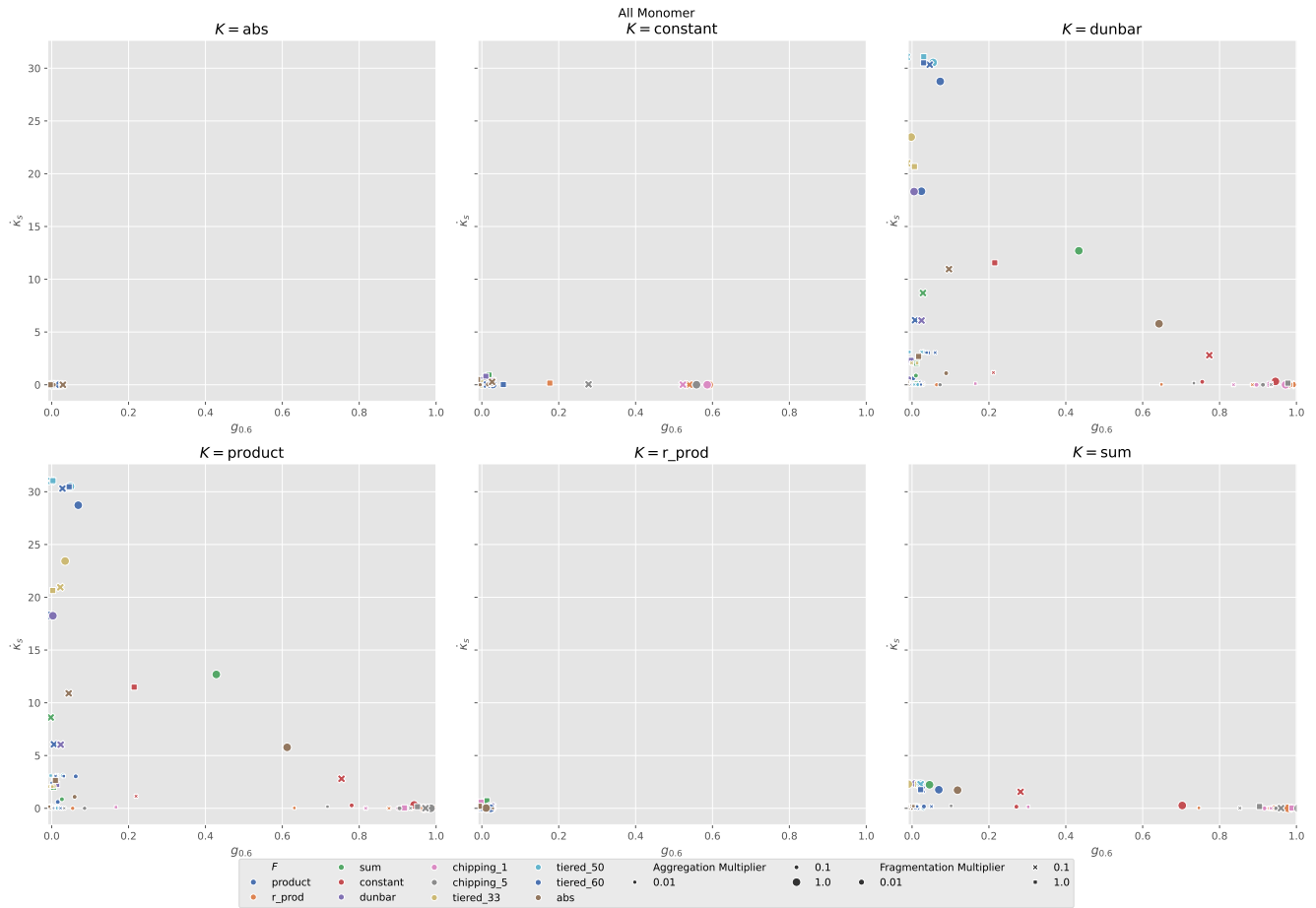


Figure 3. Cost comparison plot for all six aggregation kernels. Absolute Difference and Reverse Product are uninteresting, while Dunbar and Product are functionally the same.